Qwen

Architecting Constellation Routing: A Machine-Readable Spine for Unambiguous Artifact Generation

Architecting Unambiguous Routing: The Four-Layer Foundation

The fundamental challenge addressed by the development of a machine-readable spine index is the elimination of ambiguity in artifact generation and routing across the distributed HorrorPlace VM-constellation . As the constellation expands, encompassing multiple repositories, tiers, and specialized tools, the risk of AI-generated content being misdirected—placed in the wrong repository, assigned an invalid schema, or operating outside its designated scope—becomes a critical operational hazard . This problem is particularly acute when engaging with external AI-chat platforms that lack access to the full, private context of the constellation's internal doctrines and repository structures . The proposed solution is not a single tool but a sophisticated, multi-layered architectural pattern designed to create a deterministic and verifiable path from a high-level authoring intent to a single, correct file in a single, correct repository . This foundation rests upon four interlocking layers: manifests as the ground truth router, the spine index as the contract authority, envelopes as provenance carriers, and schemas as the final enforcement mechanism. Together, they form a closed-loop system that transforms procedural rules into programmatically enforced contracts, ensuring that "one request → one correct file in the correct repo" becomes the only predictable outcome .

The first and most foundational layer is the use of per-repository manifests as the ultimate authority for all routing decisions . These manifest documents, typically named `repo-manifest.hpc..json`, serve as the canonical source of truth for a given repository's policies and capabilities . They explicitly define which `objectKind`s (e.g., `moodContract`, `seedContractCard`) the repository is permitted to host, its governance `tier` (e.g., T1, T2, lab), default storage path patterns, and a suite of behavioral policies . Crucially, the manifest also dictates how the constellation's central routing algorithm operates. Given a pair of `(objectKind, tier)`, the algorithm performs a deterministic scan of all available manifests, selecting the first one whose rules accept the request . This ordered evaluation guarantees that for any valid

`(objectKind, tier)` pair, there is either a unique target repository or a clear routing error (such as `RouteNotFound` or `TierViolation`) . By making the manifest the sole arbiter of routing, this layer prevents "AI route drift," where a model might otherwise guess or infer a destination that contradicts established doctrine . The manifest's `policies` object further hardens this layer by enforcing rules like `oneFilePerRequest`, which mandates that a single authoring action must produce at most one primary artifact, and `requireDeadLedgerRef`, which ties the production of certain artifacts to attestation and proof mechanisms governed by the Dead-Ledger Network . This manifest-driven approach provides the initial, coarse-grained partitioning of responsibility across the constellation, establishing a stable basis upon which higher levels of abstraction can be built.

Building upon the routing stability provided by manifests, the second layer is the spine index itself, which formalizes the relationships between abstract contract families and their concrete implementation details . The spine index acts as a machine-readable catalog, consolidating information about which invariants and metrics apply to which contract types, and mapping those types to their designated repositories . For example, a request to create a "Tier-1 mood contract" would be resolved by consulting the spine index, which would return a unique triple: `(objectKind=moodContract, tier=T1, repo=Horror.Place)` . This layer directly addresses the need for clarity in contract definitions, moving beyond human-readable prose to a structured, versioned, and programmatically inspectable format . It serves as the bridge between the high-level intent of the author (or AI agent) and the low-level technical requirements of the system. The spine index is not merely a lookup table; it is a formal contract that defines the boundaries and responsibilities of each component within the constellation. It ensures that every new contract family is registered with its valid tiers, its governing invariants and metrics, and its home repository, thereby preventing orphaned or ambiguously routed artifacts before they are even generated . This layer elevates the concept of a "contract family" from a conceptual grouping into a formally recognized entity within the constellation's architecture.

The third layer introduces the principle of provenance through standardized envelopes. Every artifact produced within the constellation, whether it is a primary contract, a registry entry, or a piece of telemetry data, must travel with a metadata payload that describes its origin, intent, and validity . These envelopes, such as the `ai-authoring-request/response` forms or a generic `PrismEnvelope`, are explicitly designed to carry essential routing information, including the `targetRepo`, the `schemaRef` it conforms to, its `tier`, and the bands of invariants and metrics it represents . This redundancy is a deliberate design choice intended to make every artifact self-validating. When a CI pipeline processes an artifact, it can verify that the metadata carried within

the envelope matches the artifact's content and its location within the repository structure . If an artifact was generated for `Atrocity-Seeds` but is found in `Horror.Place`, the validation will immediately fail because the `targetRepo` field in its envelope will not match the actual repository path . This principle extends to cross-repository data flows, where NDJSON shards or QPU envelopes repeat key metadata fields like `repoName`, `tier`, invariant/metric bands, and entitlement profiles on every line of data . This ensures that orchestrators can route and validate data streams without needing to infer context from surrounding files, treating each shard or line as a standalone, verifiable unit . Envelopes thus provide the crucial link between the static contracts of the spine and the dynamic reality of artifact movement, ensuring that the integrity of the routing decision is maintained throughout the entire lifecycle of the artifact.

Finally, the fourth and lowest layer is the enforcement mechanism provided by JSON Schema. Every file generated within the constellation must conform to a specific schema that defines its structure, fields, and constraints . This layer is where the rubber meets the road, applying the rules defined in the upper layers at the granular level . The doctrine of "tight contracts," enforced by the `additionalProperties: false` keyword in JSON Schema, is paramount here . This constraint ensures that an object cannot contain any fields other than those explicitly defined in its schema, preventing accidental or malicious introduction of new, undefined data that could disrupt downstream systems [10]. The AI-chat discipline is built around respecting these schemas: models are trained to follow existing naming conventions for identifiers (e.g., `BTILE.<em>`, `BGEOM.</em>`), to use only the canonical set of invariant and metric names defined in the spine (e.g., CIC, DET, UEC, ARR), and to never invent new top-level fields . This layer provides a powerful, automated defense against structural corruption and semantic drift. CI systems can automatically validate every incoming artifact against its schema, providing immediate feedback if a contract has been violated [1] [11]. The combination of these four layers—manifests for coarse routing, the spine for contract definition, envelopes for provenance, and schemas for enforcement—creates a robust, multi-faceted architecture that systematically dismantles routing ambiguity and establishes a clear, verifiable, and scalable pathway for artifact generation across the entire HorrorPlace VM-constellation.

Defining the Spine Index: A Consolidated Contract Layer

The creation of a consolidated, machine-readable spine index is the central pillar of the strategy to achieve unambiguous artifact generation. Its purpose is to formalize the intricate web of relationships between contract families, invariants, metrics, and repository assignments into a single, authoritative JSON Schema definition and a corresponding populated instance file . This index serves as the primary reference for both human operators and AI agents, providing a definitive answer to the question of "where does this file go?" based on its type and intent . By codifying the `(objectKind, tier, repo)` triple as a first-class citizen within the constellation's architecture, the spine index moves beyond ambiguous guidelines and human interpretation, replacing them with a verifiable, programmatically accessible contract . The design of this index is guided by the need for machine-readability, consistency with existing constellations, and a clear separation of concerns between the machine-consumable core and a thin, human-readable documentation layer .

The core of the spine index is its definition of a "spine entry," which represents a formal registration of a contract family . Each entry in the index instance file would correspond to a type of artifact the constellation produces, such as `moodContract`, `eventContract`, `regionContractCard`, or the newly proposed `bci-geometry-tileset` . The JSON Schema for a spine entry would be meticulously constructed to capture all necessary metadata. It would include fields for the `objectKind` (the canonical name of the contract family), an array of allowed `tiers` (e.g., `["T1", "lab"]`), and the `repo` where artifacts of this kind must be placed . Critically, it would also contain structured references to the invariants and entertainment metrics that govern this contract family. This is achieved by linking to the canonical invariants and metrics spines, which already define the full set of valid invariants (CIC, MDI, AOS, RRM, FCF, LSG, HVF, DET, SPR, SHCI, RWF) and metrics (UEC, EMD, STCI, CDL, ARR) . The spine index entry for a `moodContract` would specify that it uses invariants like `AOS` and `SHCI` and metrics like `UEC` and `EMD`, anchoring its behavior to the established vocabulary of the constellation . Furthermore, the schema must accommodate the inclusion of mathematical constraint formulas as versioned objects, allowing for empirical refinement over time without breaking existing contracts .

To illustrate the practical application of this concept, the conversation history provides a detailed draft for a `bci-geometry-tileset-v1.json` schema, which perfectly exemplifies the principles of the spine index . This schema is not just a data structure; it is a comprehensive contract that embodies the four-layer architecture. Its `appliesTo`

object directly specifies the repository (`Horror.Place`), tier (`1`), and region family it targets, thus defining its scope within the routing triple . Its `integration` block points to runtime modules and Dead-Ledger profiles, bridging the gap between the static contract and the dynamic systems that consume it . Even more importantly, fields like `deadledgerref` and `invariants_used` explicitly tie the contract to the governance and doctrinal layers of the constellation, ensuring that any changes to BCI-related geometry are subject to attestation and safety verification . The structure of this single file demonstrates how the research goal can be realized: it formalizes the routing triple, consolidates contract families, and integrates constraints into a single, cohesive, and machine-readable document .

The implementation of the spine index follows a disciplined, two-step process. First, a JSON Schema (`spine-index-schema-v1.json`) is created to define the structure of a spine entry, including all required properties, data types, enums, and constraints . Second, an instance file (`spine-index-instance.json`) is populated with the actual entries for all known contract families . This instance file becomes the ground truth that tools and AI agents consult to resolve routing requests. To support external AI platforms that lack access to private repositories, a thin, human-readable documentation layer can be generated from this machine-readable core . This documentation would not replace the schema but would summarize its contents in a more digestible format, explaining the role of each repository, the meaning of key invariants and metrics, and providing examples of valid requests . This dual-layer approach ensures that both machines and humans operate from the same authoritative source, minimizing the risk of misinterpretation. The discipline of creating a single, consolidated file aligns with the existing constellation doctrine of having a clear, singular point of truth for any given concept . The following table outlines the key components of the proposed spine index structure.

Component	Description	Example Value
`$schema`	The JSON Schema version being used.	`"https://json-schema.org/draft/2020-12/schema"`
`$id`	The unique identifier for the spine index schema.	`"https://horror.place/schemas/spine-index-v1.json"`
`title`	A human-readable title for the schema.	`"HorrorPlace VM Constellation Spine Index"`
`description`	A detailed explanation of the schema's purpose.	`"Consolidated index formalizing the (objectKind, tier, repo) triple..."`
`type`	The data type of the root element.	`"object"`
`additionalProperties`	Prevents unknown properties at the root level.	`false`
`required`	Lists the mandatory top-level properties.	`["$defs", "spineEntries"]`
`properties`	Defines the structure of the root object.	Contains definitions for `spineEntries` and `$defs`.
`spineEntries`	An array of objects, each representing a contract family.	`[{"objectKind": "moodContract", "tier": "T1", "repo": "Horror.Place", ...}]`
`$defs`	Defines reusable sub-schemas for complex types.	Contains definitions for `InvariantBand`, `MetricBand`, etc.

This structured approach ensures that the spine index is not only a repository of information but a formal, validated contract that can be consumed by a wide range of tools, from CI/CD pipelines that perform automated validation [1] to AI agents that need to determine the correct `targetRepo` for a generation request . By making the routing logic explicit, versioned, and machine-enforceable, the spine index provides the critical foundation needed to scale the constellation's operations reliably and safely.

Manifests as the Ground Truth Router: Codifying Repository Policies

While the spine index provides a high-level catalog of contract families and their properties, the per-repository manifest serves as the ultimate, ground-truth router for all artifact placement decisions . These manifest documents are the constitution of each repository, defining its role, permissions, and policies within the VM-constellation . The strategic importance of manifests lies in their ability to transform a potentially ambiguous routing process into a deterministic and provably correct algorithm . By creating a formal

JSON Schema for these manifests, their logic becomes machine-verifiable, enabling automated tools to resolve routing conflicts and ensure that every artifact finds its way to its designated home without relying on human or AI intuition . This layer is the bedrock upon which the rest of the architectural strategy is built, as it provides the definitive answer to "which repositories accept which kinds of files?".

The proposed `repo-manifest-v1.json` schema provides a comprehensive blueprint for codifying these policies . It organizes all critical routing and governance information into a structured, validated format. At its core, the manifest defines the repository's identity through its `repoName` and `tier` . The `allowedObjectKinds` array explicitly lists the canonical identifiers for the types of primary artifacts this repository is authorized to host, such as `["seedContractCard"]` for the Atrocity-Seeds repository . This creates a strict whitelist, preventing the accidental ingestion of files that do not belong. Complementing this is the `schemaWhitelist`, which restricts the set of permissible schema references (`schemaRef`) that can be used within the repository, further tightening control over the data's structure . For example, a manifest might allow only schemas prefixed with `https://horror.place/schemas/tier1/`.

Beyond simple allowances, manifests incorporate a rich set of policies and constraints that dictate the expected behavior of authors and tools . The `policies` object contains boolean flags like `oneFilePerRequest`, which enforces a strict "one request, one file" discipline, and `requireDeadLedgerRef`, which mandates that artifacts destined for certain tiers must be backed by cryptographic proofs managed by the Dead-Ledger Network . Other policies, such as `maxFileSizeBytes` and `forbiddenFields`, act as guardrails against resource abuse and structural corruption . The manifest also defines the repository's physical layout through the `storage` object, which contains `defaultPaths`—patterns for where files of different kinds should be stored, often scoped by phase or tier . Finally, the `routing` section provides hints for the resolver, including `idPrefixes` that identify ownership of certain contract IDs (e.g., `EVT.` for events) and an optional `priority` field that influences the manifest scanning order . The formalization of this entire structure into a JSON Schema is a critical step toward creating a mathematically rigorous understanding of the constellation's topology .

With a formal schema in place, the routing algorithm itself can be treated as a mathematical function. Let the set of all manifests be $M=\{m_1,m_2,...,m_n\}$. Each manifest m_i defines a predicate $A_i(k,t)$, which is true if an `objectKind` k and `tier` t are accepted by that manifest's rules . The routing function $R(k,t)$ can then be defined as a piecewise function that returns the manifest m_j that wins the routing contest, or $\perp$ (failure) if no manifest accepts the request .

$$R(k,t) = \begin{cases} m_j & \text{where } j = \min i \mid A_i(k,t) \wedge \neg \text{implicitDeny}_i \\ \perp & \text{if no such } i \end{cases}$$

This formulation makes the algorithm's behavior explicit and testable. Research objectives can now focus on proving properties of this function, such as non-ambiguity (for any given (k,t), there is at most one winning manifest) and totality (for all (k,t) in the constellation's scope, $R(k,t)$ yields a result) . By writing a generator script that reads all manifests and builds a `repo-routing-index-v1.json`, one can create a thin, public-facing façade that exposes this complex logic in a simplified form, mapping `objectKind -> defaultRepo` and `idPrefix -> repoName` . This index is invaluable for external AI platforms, giving them the necessary information to route requests correctly without needing access to the full constellation's private manifest set . Ultimately, codifying the manifest schema turns the routing layer from a collection of human-written policies into a formal system that can be analyzed, proven, and executed with absolute precision, forming the reliable foundation required for a large-scale, AI-assisted development environment.

Envelopes and Schemas: Ensuring Provenance and Structural Integrity

While manifests and the spine index establish the "who" and "what" of artifact generation, the layers of envelopes and schemas are responsible for guaranteeing the "where" and "how"—ensuring that artifacts are placed in the correct repository and adhere to the precise structural and semantic rules defined by their contracts. This is achieved through a dual strategy of embedding provenance information directly within the artifacts themselves via standardized envelopes and enforcing those rules at the most granular level using JSON Schema validation . This approach creates a system of redundant checks where every piece of data carries its own credentials, allowing for independent, isolated validation by CI pipelines and orchestrators, thereby eliminating ambiguity and reinforcing the routing decisions made at higher architectural layers . This practice of "fat, standalone shards" is a cornerstone of the constellation's design philosophy, ensuring that context is never silently reinterpreted .

The principle of embedding provenance is realized through various envelope formats, most notably the `ai-authoring-request` and `ai-authoring-response` schemas . These wrappers are not mere containers; they are integral parts of the contract. An `ai-authoring-request` must contain fields like `objectKind`, `tier`, `schemaRef`, and,

critically, `targetRepo` . Before an AI agent is permitted to generate a file, it must first resolve these fields, ideally by consulting the manifest router and spine index . Once a file is generated, the `ai-authoring-response` wraps the file's content along with this same metadata, plus diagnostic information . This structure ensures that the metadata and the content are always treated as a single unit. The CI pipeline can then validate that the `targetRepo` specified in the response actually corresponds to the repository where the file is being committed, and that the file's content conforms to the schema referenced by `schemaRef` . This prevents a scenario where an AI agent generates a file for one repository but mistakenly places it in another. This discipline mirrors the `oneFilePerRequest` policy enforced by manifests, extending it from a procedural rule to a structural requirement .

The `PrismEnvelope` serves a similar purpose for artifacts that require deep provenance tracking, such as those interacting with the Dead-Ledger Network . It embeds metadata about the artifact's origin, dependencies (`referencedIds`), and its relationship to cryptographic proofs (`prismMeta`, `deadledgerref`) . By making this information self-contained, the system can trace an artifact's lineage and verify its compliance with governance policies at any point in its lifecycle, regardless of its current location in the constellation . This is particularly important for assets that affect player state and safety, as it allows for audits and reviews based on the exact conditions under which the asset was created and used. The `zkpproof-envelope-shell-v1` schema further reinforces this by forcing proof envelopes to carry a complete set of tier, invariant, metric, and safety metadata, stripping out any raw, sensitive data like trauma information and retaining only the contracted, summarized state . This ensures that even the evidence of compliance is structured, verifiable, and safe.

At the lowest level, JSON Schema acts as the final and most granular gatekeeper, enforcing the structural integrity of every artifact . The doctrine of "tight contracts," implemented using `additionalProperties: false`, is the linchpin of this enforcement layer . This keyword ensures that an object adheres strictly to its defined schema, rejecting any extra, unintended fields. This is a critical defense against schema evolution issues and accidental data corruption [10]. The AI-chat discipline is built around respecting these schemas: models are instructed to use existing ID patterns (e.g., `BTILE.*` for tilesets), to only reference the canonical set of invariants and metrics defined in the spine, and to never invent new top-level fields . For example, a `bci-geometry-tileset` schema would define `bciBands` with a fixed enum of `["UNDERSTIMULATED", "OPTIMAL_STRESS", "OVERWHELMED"]`; an attempt by an AI to generate a tileset with a new, invalid mode would be rejected by the schema validator . The validation process is systematic: "schema → invariants/metrics → manifest → envelope/provenance" . An artifact that passes schema validation has

demonstrated that it is structurally sound. One that passes manifest validation has proven it is destined for an appropriate repository. And one that passes envelope validation has confirmed its own provenance. Only artifacts that successfully navigate this entire chain of validation are considered valid and can proceed to be applied to the constellation. This multi-stage validation pipeline, powered by machine-readable schemas and enforced by CI/CD systems like Jenkins or GitHub Actions, provides a robust, automated, and highly reliable mechanism for maintaining the integrity of the entire system [4] [5].

Governing Runtime Behavior: From Static Contracts to Dynamic Map Functions

The machine-readable contracts defined in the spine index and manifests are not merely administrative artifacts; they are blueprints for dynamic, runtime behavior within the HorrorPlace VM-constellation. The transition from a static, declarative contract to a live, procedural function is managed by a carefully designed adapter layer within the runtime engine, referred to as Codebase-of-Death in the provided context . This layer acts as the interpreter, taking the pure data of a contract—such as `bci-geometry-tileset.v1.json`—and realizing it as a concrete, functional map function that alters the game world in real-time . This decoupling of contract from implementation is a core architectural strength, as it allows the creative and safety-critical logic to evolve independently of the underlying engine, governed entirely by the attested rules in the `Dead-Ledger-Network` . This ensures that any change to a BCI-aware tileset, for instance, is not a code change but a contract change, which can be reviewed, tested, and verified before deployment.

The workflow for this runtime realization begins with an adapter module in the engine, likely written in a scripting language like Lua, that is responsible for loading and interpreting the relevant contracts for a given session . For the `bci-geometry-tileset` example, this adapter would load the JSON file defining the tileset, parse its `geometryProfiles`, and build an efficient in-memory index . This index would be keyed by a composite of selection criteria found in the profile's `selector` object: `(regionIdPattern, bciMode, tier, fearBand)` . At runtime, when the game logic needs to select a tile profile—for example, during a pacing phase shift or in response to a BCI signal—the adapter invokes a narrow API, such as `MapF.selectTileProfile(regionId, context)`, passing in the current state of the game . The `context` object provides the adapter with all the necessary inputs: the invariant bands from the `H.` namespace (CIC, LSG, DET, etc.), the normalized BCI state

from the `BCI.` namespace (intensity mode, fearindex band, overload flag), and the effective entitlement tier from the `Policy.DeadLedger` token .

The adapter's matching logic then performs a deterministic lookup against its pre-built index. It searches for the best `geometryProfile` that satisfies all constraints: the profile must be applicable to the current `regionId`, its `bciMode` must match the live BCI intensity mode, its `tier` must be less than or equal to the entitlement tier granted by the `DeadLedger`, and its `targets` for invariants like `DET` and `SPR` must not be violated . This matching process is the direct execution of the contract. If the BCI state enters `OVERWHELMED` mode, the logic can enforce downshifts by preferring profiles with lower safety caps, as specified in the `bciSafety` block of the contract . Once a matching profile is selected, the adapter proceeds to the "apply" stage. It retrieves the `cameraGrammarRef` and `lightingStyleRef` from the profile and passes them to the appropriate subsystems—the camera controller and the lighting engine—to realize the contracted visual and gameplay state . This entire process keeps the engine agnostic to the specifics of the BCI device; it only interacts with the normalized, contracted state provided by the BCI adapter, demonstrating the power of treating hardware interfaces as adapters to a standardized contract .

This contract-driven approach extends seamlessly into the research and tuning loop. Telemetry systems are designed to capture the `currentTilesetId` and `geometryProfileId` alongside other state data, allowing researchers at `Neural-Resonance-Lab` to analyze how each specific profile affects player experience metrics like UEC, EMD, and STCI, as well as physiological states like time spent in `OVERWHELMED` mode . The findings from this analysis are not used to patch the engine's code. Instead, they lead to updates in the static contracts themselves. Researchers might propose a new `geometryProfile` with refined `invariant` and `metric` bands, or adjust the `bciSafety` thresholds to improve safety or efficacy. These updated contracts are then submitted for attestation by the `Dead-Ledger-Network` . The attestation process verifies that the proposed changes, when combined with the `bcistate/sessionsafety proofs`, keep the overall risk exposure within acceptable limits . Once attested, the new contract version is deployed, and the runtime map functions automatically begin using the improved logic on their next lookup. This creates a continuous, empirically-driven feedback loop where the engine remains static and stable, while the emergent behavior and content are constantly refined and optimized through the evolution of the contract layer. The map functions are simply a contract-driven lookup over a vast library of empirically tuned tilesets, governed by the `Horror.Place` contract authority and the `Dead-Ledger-Network` governance layer .

Synthesis and Strategic Implementation Pathway

The collective body of research and discussion culminates in a clear and actionable strategy for establishing a robust, scalable, and unambiguous artifact generation system for the HorrorPlace VM-constellation. The core directive is to formalize the routing triple `(objectKind, tier, repo)` into a machine-readable spine index, which serves as the central nervous system for coordinating activity across the entire distributed ecosystem . This initiative is not merely a technical enhancement but a strategic imperative to mitigate the risks of operational drift, especially when engaging with external AI platforms that lack visibility into the constellation's private repositories . The proposed solution is not a monolithic tool but a layered architectural framework composed of manifests, the spine index, envelopes, and schemas, each playing a distinct yet interconnected role in enforcing a deterministic and verifiable workflow from intent to execution . The synthesis of these materials reveals a mature architectural vision where procedural rules are elevated to programmatically enforced contracts, ensuring that the constellation can grow in complexity without sacrificing reliability or safety.

The proposed implementation pathway prioritizes stability, machine-readability, and a phased rollout that builds confidence at each stage. The immediate priority is the consolidation of the spine index, which will serve as the primary interface for resolving high-level authoring requests . The first concrete step is to create the `spine-index-schema-v1.json` and populate the `spine-index-instance.json` file . This document must precisely define the structure of a "spine entry," capturing the essential metadata: `objectKind`, allowed `tiers`, designated `repo`, and mappings to the relevant invariants and metrics . It should also include a placeholder structure for key constraint formulas, such as the calculation for `ARR` or the caps on `DET` based on `AOS` and `CIC`, to be filled in as empirical data becomes available . This spine index becomes the canonical source of truth for routing, providing a definitive answer to where any given contract family belongs.

Concurrently, the foundational routing layer must be hardened by formalizing the repository manifests. Creating the `schemas/core/repo-manifest-v1.json` schema, based on the provided draft, is a critical prerequisite . This codifies the ground-truth router, transforming the manifest from a human-readable document into a machine-verifiable contract. With this schema defined, a routing index generator can be developed to create a `repo-routing-index-v1.json` file, which simplifies the complex manifest logic into straightforward lookups for both human and AI consumption . This public-facing index is particularly vital for enabling external AI platforms to participate in the constellation's creation process correctly, as it provides them with the necessary routing logic without exposing private repository structures .

The final step in this foundational work is to formalize the AI-chat discipline into a comprehensive skill profile and a set of strict validation rules that mandate adherence to the manifest, spine index, and schema contracts before any file is generated or accepted .

Looking forward, the long-term success of this system depends on its ability to evolve and adapt. The current framework is built on static, pre-defined contracts, but the future will inevitably involve more dynamic content and runtime-generated logic. The `bci-geometry-tileset` example provides a template for this, showing how static contracts can drive dynamic runtime behavior through a dedicated adapter layer . The next frontier is to generalize this pattern, creating a flexible framework that allows for the controlled evolution of contracts and the safe deployment of new behaviors governed by the same principles of attestation and verification . The spine index must be designed to accommodate future versions of constraint formulas (e.g., `ARR_v1`, `ARR_v2`) to support this empirical refinement . By establishing this robust, contract-first architecture today, the HorrorPlace VM-constellation can confidently embrace the future of AI-assisted development, ensuring that creativity and innovation proceed hand-in-hand with the uncompromising rigor of machine-enforced contracts.

Reference

1. JSON Schema报文结构自动化验证方案 - OSCHINA - 开源× AI https://my.oschina.net/emacs_8002834/blog/19484073
2. [PDF] Managed integrations for AWS IoT Device Management https://docs.aws.amazon.com/pdfs/iot-mi/latest/devguide/iot-mi-dg.pdf
3. JSON Schema Validation in Python | PDF - Scribd https://www.scribd.com/document/660484250/Python-Jsonschema-Readthedocs-Io-en-Latest
4. GitHub Actions vs. Jenkins: Which one's right for your team? | Buildkite https://buildkite.com/resources/ci-cd-perspectives/github-actions-vs-jenkins-which-one-s-right-for-your-team/
5. Which CICD tool should I use? - Stack Overflow https://stackoverflow.com/beta/discussions/77102055/which-cicd-tool-should-i-use
6. Investigating the Evolution of Resilient Microservice Architectures https://www.mdpi.com/2673-6470/5/3/27
7. [PDF] IBM watsonx.data: Release V2.0.0 https://www.ibm.com/support/pages/system/files/inline-files/IBM%20watsonx.data%20version%202.0.0.pdf

8. [PDF] PROCEEDINGS - ResearchGate https://www.researchgate.net/profile/Mamyrbayev_Orken/publication/333435792_The_Influence_of_Various_Text_Characteristics_on_the_Readability_and_Content_Informativeness/links/5d0dd7c4299bf1547c73c8d9/The-Influence-of-Various-Text-Characteristics-on-the-Readability-and-Content-Informativeness.pdf
9. [XML] https://mirrors.aliyun.com/rockylinux-vault/8.4/BaseOS/source/tree ... https://mirrors.aliyun.com/rockylinux-vault/8.4/BaseOS/source/tree/repodata/ed62329558715b6b58e23b65f1cb746105f8f339f359f82e23d2c328dd932b96-other.xml.gz
10. Cross-Stage Vulnerabilities in Large Language Model Architectures https://arxiv.org/html/2510.27190v1
11. Establishing a Data Mesh in a Fractal Enterprise - LinkedIn https://www.linkedin.com/pulse/establishing-data-mesh-fractal-enterprise-ensuring-quality-selman-mywfc
12. I built an AI agent with Claude Code that doesn't leak sensitive data ... https://www.linkedin.com/posts/danielwalshireland_i-built-an-ai-agent-with-claude-code-that-activity-7429150292501225472-j5kR
13. Chapter 2. OpenShift CLI (oc) | CLI tools - Red Hat Documentation https://docs.redhat.com/en/documentation/openshift_container_platform/4.13/html/cli_tools/openshift-cli-oc
14. kubernetes.client.models package https://k8s-python.readthedocs.io/en/stable/kubernetes.client.models.html
15. Glossary - Kubernetes https://kubernetes.io/docs/reference/glossary/
16. Chapter 5. Developing Operators | OpenShift Container Platform | 4.9 https://docs.redhat.com/en/documentation/openshift_container_platform/4.9/html/operators/developing-operators
17. Agent Behavioral Contracts: Formal Specification and Runtime ... https://arxiv.org/html/2602.22302
18. MMTU: A Massive Multi-Task Table Understanding and Reasoning ... https://openreview.net/forum?id=ryUzgwD6UQ&referrer=%5Bthe%20profile%20of%20Yeye%20He%5D(%2Fprofile%3Fid%3D~Yeye_He1)
19. [PDF] Towards a Service-Based Internet (Lecture Notes in Computer ... https://www.researchgate.net/profile/Paulo-Simoes-4/publication/220864225_MICIE_An_Alerting_Framework_for_Interdependent_Critical_Infrastructures/links/0c960531ced062d916000000/MICIE-An-Alerting-Framework-for-Interdependent-Critical-Infrastructures.pdf?origin=scientificContributions
20. DevOps - SAP Community https://community.sap.com/t5/ca-erp/DevOps/pd-p/51112e3c-4b78-4058-a637-67f453c196c9

21. Intelligent Software Methodologies, Tools and Techniques https://link.springer.com/content/pdf/10.1007/978-3-319-17530-0.pdf
22. [EPUB] SITUATED COGNITION AND ITS CRITICS - Frontiers https://public-pages-files-2025.frontiersin.org/research-topics/11500/epub
23. (PDF) Manual Power Designerdata modeling - Academia.edu https://www.academia.edu/7673215/Manual_Power_Designerdata_modeling
24. Theory and Applications of Ontology PDF - Scribd https://www.scribd.com/document/439984121/Theory-and-Applications-of-Ontology-pdf
25. 平台和工具 - 架构师 https://architect.pub/book/export/html/36
26. Debian -- Software Packages in "sid", Subsection python https://packages.debian.org/en/sid/python/
27. 200 Python Interview Questions & Answers | PDF - Scribd https://www.scribd.com/document/976641328/Prep-kit
28. Scala - Springer Nature https://link.springer.com/content/pdf/10.1007%2F978-3-319-77525-8_245.pdf
29. “Lar, doce lar”: Os contratos sobre a habitação familiar Algumas ... https://www.researchgate.net/publication/342469644_Lar_doce_lar_Os_contratos_sobre_a_habitacao_familiar_Algumas_particularidades_de_regimeProtection_of_the_family_home_and_contract_law
30. ASPLOS '25: Proceedings of the 30th ACM International Conference ... https://dl.acm.org/doi/proceedings/10.1145/3669940?tocHeading=heading1
31. Open Source Projects - DaoCloud Enterprise https://docs.daocloud.io/native/open/
32. Computer Science - arXiv https://www.arxiv.org/list/cs/new?skip=300&show=1000
33. Experts for Force Refresh Cluster State - Linknovate https://www.linknovate.com/search/?query=force%20refresh%20cluster%20state
34. 2022 Review of Data-Driven Plasma Science - IEEE Xplore https://ieeexplore.ieee.org/iel7/27/10225460/10214236.pdf
35. From Kubernetes Gatekeeper to Full-Stack Governance with OPA https://www.pulumi.com/blog/kubernetes-gatekeeper-full-stack-governance-opa/
36. SemLoc: Structured Grounding of Free-Form LLM Reasoning ... - arXiv https://arxiv.org/html/2603.29109v1
37. Graphylo: A deep learning approach for predicting regulatory DNA ... https://pmc.ncbi.nlm.nih.gov/articles/PMC10867641/
38. BOAD: Discovering Hierarchical Software Engineering Agents via ... https://arxiv.org/html/2512.23631v2

39. Virtual Screening Strategy to Identify Retinoic Acid-Related Orphan ... https://pmc.ncbi.nlm.nih.gov/articles/PMC10145393/
40. Traversal-as-Policy: Log-Distilled Gated Behavior Trees as ... - arXiv https://arxiv.org/html/2603.05517v1
41. Learnable latent embeddings for joint behavioural and neural analysis https://pmc.ncbi.nlm.nih.gov/articles/PMC10172131/
42. Chapter 1. Introduction to Apicurio Registry - Red Hat Documentation https://docs.redhat.com/en/documentation/red_hat_build_of_apicurio_registry/2.6/html/apicurio_registry_user_guide/intro-to-the-registry_registry
43. [PDF] 13071118_DAFFODIL-260824-2226-198342.pdf https://issues.apache.org/jira/secure/attachment/13071118/13071118_DAFFODIL-260824-2226-198342.pdf
44. Amazon Interview Insights and Questions | PDF - Scribd https://www.scribd.com/document/338829928/Scribd-Ddd
45. Conceptual Modeling - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-031-47262-6.pdf
46. Intelligent Computing and Automation - Springer Nature https://link.springer.com/content/pdf/10.1007/978-981-96-0143-1.pdf